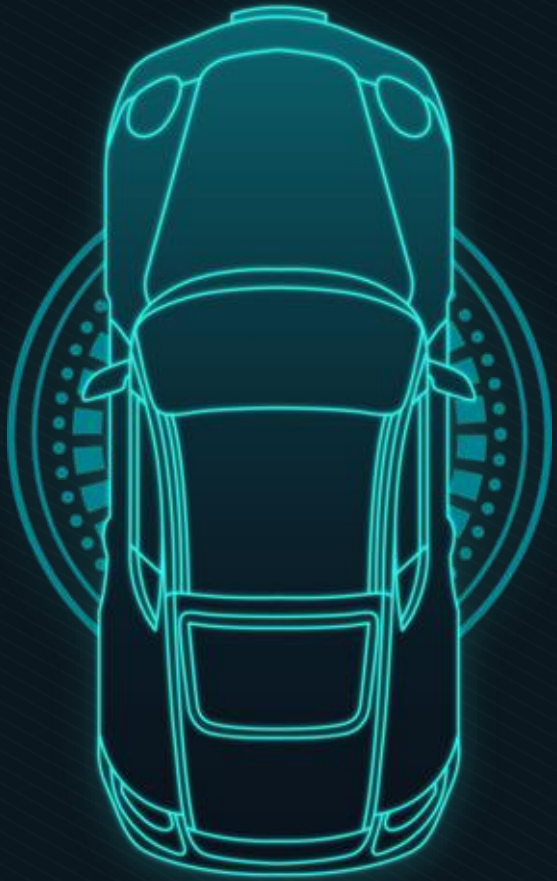
REAL-TIME DRIVER DROWSINESS DETECTION SYSTEM



TEAM MEMBERS

01

Sumit Sharma

NO.: 20233279

02

Sourabh Kumar

NO. 20233353

03

Swatantra Kumar Maurya

NO. 20233285

04

Vishwajeet Gupta

NO. 20233316

05

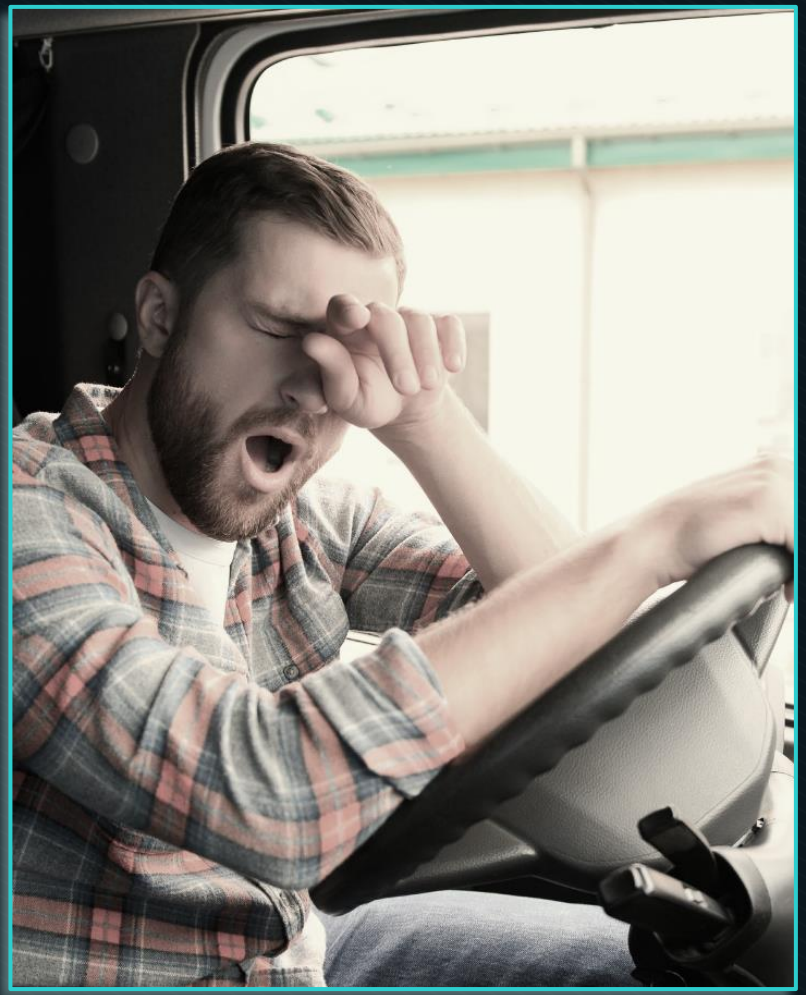
Viswa Ravindren

NO. 20233584

INTRODUCTION

- Driver drowsiness is a major road safety concern that reduces attention, concentration, and reaction time.
- Drivers may experience prolonged eye closure, frequent blinking, and reduced alertness.
- Computer vision enables real-time monitoring of visible facial cues using a camera.
- This project aims to develop a system that detects possible drowsiness and alerts the driver through an audible warning.
- The system uses webcam input and processes video locally.



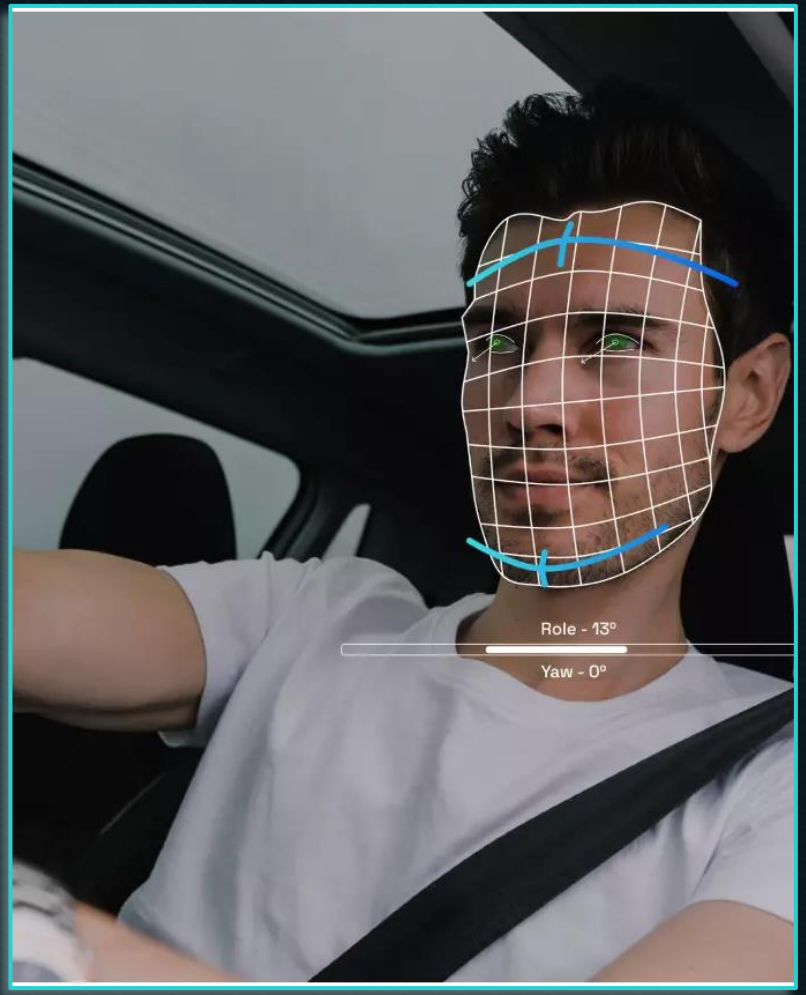


PROBLEM STATEMENT

- Drowsiness can impair a driver's ability to respond to changing road conditions.
- Drivers may not recognize their own declining alertness.
- Manual monitoring of driver alertness is difficult to maintain continuously.
- Traditional safety measures may not provide a direct warning based on the driver's current facial cues.
- Therefore, an automated, low-cost system is proposed to detect prolonged eye closure and generate timely alerts.

OBJECTIVES

- Capture real-time video using a webcam.
- Detect facial landmarks and identify eye regions.
- Calculate the Eye Aspect Ratio (EAR) to estimate eye openness
- Detect prolonged eye closure using a time-based algorithm.
- Generate visual warnings and audible alarms.
- Develop a web-based monitoring interface using Flask.
- Evaluate the system under different lighting conditions and user scenarios.



PROPOSED SYSTEM

Working Process:

1. Capture live video through a webcam.
2. Process video frames using OpenCV.
3. Detect facial landmarks using MediaPipe.
4. Calculate the Eye Aspect Ratio (EAR).
5. Monitor eye closure over time.
6. Identify possible drowsiness when a configured threshold and duration are exceeded.
7. Trigger an audible alarm and display a warning on the dashboard.

The initial version uses pretrained facial landmark detection and a rule-based algorithm without training a custom model.

TECHNOLOGY STACK

Python:	Main programming language used to implement the detection algorithm.
OpenCV	Captures webcam video and performs image processing.
MediaPipe	Detects facial landmarks around the eyes using a pretrained model.
NumPy	Performs numerical calculations for eye measurements.
Flask	Provides the Python web backend and connects the detection system to the dashboard.
HTML, CSS and JavaScript:	Build the frontend interface and interactive controls.
Pygame	Plays the audible warning when possible drowsiness is detected.

SYSTEM ARCHITECTURE



Webcam / Video Input

OpenCV — Video Frame Processing

MediaPipe — Facial Landmark Detection

Eye Aspect Ratio (EAR) Calculation

Time-Based Drowsiness Detection



Decision-Making Module

Flask Web Dashboard + Audible Alarm

- Input Layer: Captures live video.
- Processing Layer: Detects facial landmarks and calculates eye measurements.
- Detection Layer: Evaluates eye closure duration and determines the current state.
- Alert Layer: Generates visual and audible warnings.
- Presentation Layer: Displays the live feed and detection status through the web interface.

EYE ASPECT RATIO (EAR)

The Eye Aspect Ratio is a geometric measurement used to estimate eye openness from facial landmarks.

$$\mathrm{EAR} = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|}$$

Where:

- P1 and P4 represent horizontal eye landmarks.
- P2, P3, P5 and P6 represent vertical eye landmarks.
- Vertical distances measure the height of the eye.
- Horizontal distance measures the width of the eye.

Interpretation

- Open eyes generally produce a higher EAR.
- Closed eyes generally produce a lower EAR.
- If EAR remains below a calibrated threshold for a specified duration, the system flags possible drowsiness.

EAR thresholds require calibration and testing for different users and conditions.

DROWSINESS DETECTION ALGORITHM

1. Capture a frame from the webcam.
2. Detect the driver's face and facial landmarks.
3. Locate the eye landmarks.
4. Calculate EAR for both eyes.
5. Compare the EAR values with a configurable threshold.
6. Track the duration of continuous eye closure.
7. If eye closure exceeds the configured duration, classify the state as possible drowsiness.
8. Trigger an audible alarm and update the dashboard.
9. Reset the closure timer when the eyes reopen.



Detection States:



NORMAL

Eyes appear open or normal blinking is detected.



DROWSY

Prolonged eye closure meets the detection criteria



WARNING

Eye closure or another configured warning condition persists.



FACE NOT DETECTED

The face cannot be reliably tracked.

Features

- Live webcam feed.
- Facial landmarks displayed around the eyes.
- Real-time EAR values.
- Current driver status.
- Eye-closure duration monitoring.
- Audible warning when possible drowsiness is detected.
- Start, stop, reset and mute controls.

Alert Mechanism

- When the eye-closure duration exceeds the configured threshold, the system displays a warning and plays an alarm.
- A cooldown mechanism prevents the alarm from restarting continuously.
- The interface distinguishes missing face data from detected drowsiness.

TESTING, EVALUATION AND LIMITATIONS



Testing Scenarios

- Normal blinking and prolonged eye closure.
- Different lighting conditions.
- Users wearing glasses.
- Head movement and temporary face loss.
- Camera disconnection and invalid video frames.
- Alarm response time and system performance.



Evaluation Metrics:

- Precision, recall and F1-score on labeled test data.
- False alarm rate and missed detection rate.
- Processing speed in frames per second (FPS).
- Time required to trigger an alert.



Limitations:

- Poor lighting and eye occlusion may reduce detection reliability.
- EAR thresholds can vary between individuals.
- Eye closure alone cannot definitively establish that a driver is drowsy.
- The system is a prototype and is not certified for automotive safety use.

CONCLUSION

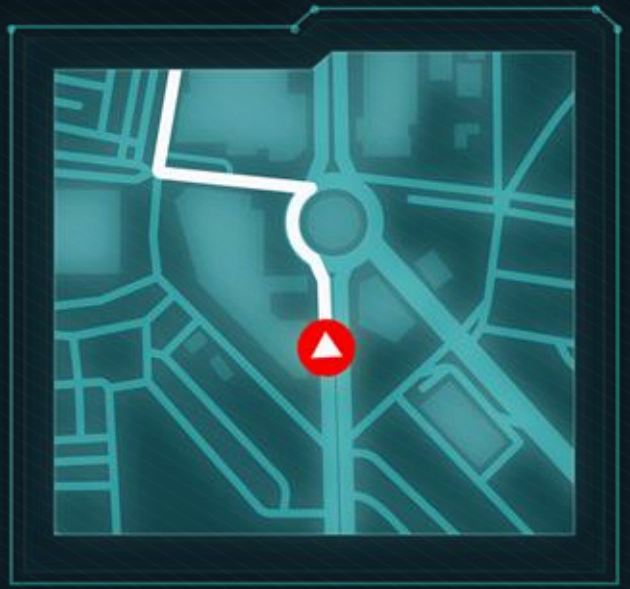


- The project proposes a real-time driver drowsiness detection system using computer vision.
- OpenCV processes video frames, while MediaPipe detects facial landmarks.
- EAR-based logic estimates prolonged eye closure.
- Flask provides the foundation for a web-based monitoring dashboard.
- The initial system can operate without collecting a dataset or training a custom model.

FUTURE SCOPE



- Integrate yawning detection.
- Add head-pose estimation.
- Explore CNN-based eye-state classification.
- Improve performance through calibration and extensive testing.
- Add detection history and session reports.
- Investigate deployment on embedded devices for in-vehicle monitoring.



THANK YOU